

TAAR: The Design of a 3D Printed Robotic Arm

Keller Hydle
kellerhydle@gmail.com

Abstract

The goal of the TAAR project was to design and engineer a full four-degrees-of-freedom (4DOF) robotic system efficiently, leveraging modern technologies such as the Rust Programming Language (for firmware applications) and 3D printed manufacturing (FDM printing). This project involved mechanical and electrical engineering, software design, control theory, applied mathematics, and kinematic physics.

TAAR is an attempt to prove robotic systems can be "boiled down" to simple mechanical and electrical components. Around 78% of TAAR is 3D printed, with the rest of the mass comprised of motors, control boards, and bearings. It was constructed over the course of three months (including off and on work) and was completed August 13th, 2026.

1 Mechanical Components

1.1 Material Choices

Fused Deposition Modeling (FDM) printing exposes a vast amount of plastic and polymer choices, some popular ones including PLA (Polylactic Acid), PETG (Polyethylene Terephthalate Glycol), and ASA (Acrylonitrile Styrene Acrylate).

Table 1: Plastic Comparison

Composition	Tensile Strength (MPa)	Elastic Modulus (GPa)	Cost (\$USD/kg)
PLA	60	3.5	15.35
PETG	50	2.1	18.56
ASA	45	2.2	24.26

While the advantages of PLA are apparent, its brittle properties made it a poor choice for moving components. ASA has a low tensile strength, and utilizing FDM-printing for ASA is a technical process (usually involving near-perfect temperature and humidity conditions to achieve high-quality parts.) PETG

provides a common ground between the two, with properties suitable for moving and static parts. PETG's elasticity was taken into consideration; its better to have a material that can bend rather than shatter when under stress or load. Additionally, extruding PETG on standard FDM-printers is a common, safe process.

1.2 Base

The base design mounts a NEMA-17 stepper motor vertically (shaft up) with a horizontal mount holding another NEMA-17 facing shaft outwards.

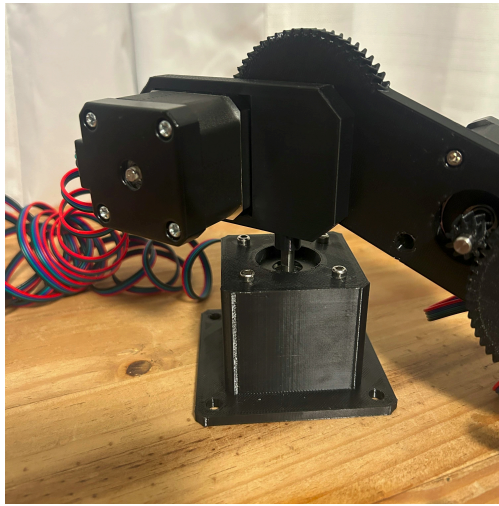


Figure 1: Base assembly

1.3 Arm Extension

There are two arm extensions with a common length of 100mm joint to joint. Each extension contains a NEMA-17 mount with a through-hole shaft for the back-mounted gear. The back-mounted gear is a helical gear embedded on the start of each arm extension. The extensions also contain slots for standard 608 bearings, allowing joint-to-joint connection.

Together, with the base and two arm extensions, TAAR can achieve full 4DOF motion.



Figure 2: Second facing arm extension connected in series

1.4 Gear Design

The back-mounted gear on the arm extension along with the motor shaft gear maintain a ratio of 6:1. The motor shaft gears are press fit, with precisely 10 teeth, whilst the back-mounted gears are extruded in the extension with 60 teeth.

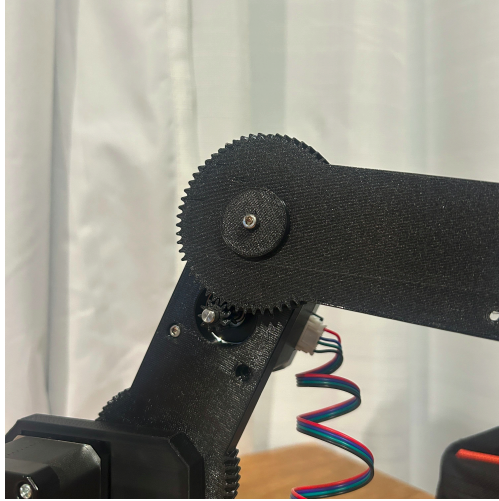


Figure 3: Press-fit shaft gear driving the arm extension back-mounted gear

2 Electrical Components

2.1 Power Distribution

Powering the electrical systems (drivers, modules, and motors) was done with a standard 24 VDC Power Supply Unit (PSU). The 120 VAC input is rectified into a stable 24 VDC. 24V was chosen due to the current governed by Ohm's Law (solving for R):

$$R = \frac{V}{I} \quad (1)$$

This low-current, high-voltage trade-off prevents the heat produced by electrical resistance from melting critical components and wiring.

2.2 Motors

NEMA-17 stepper motors became an apparent choice, given their precise motion, strong holding torque, and low power requirements. Given that stepper motors move in discrete step amounts, precise motion can be achieved down to 1.8° per step (ultra-precise motion can be achieved with micro-stepping.)

2.3 Control Devices & Drivers

STMicroelectronic’s NUCLEO-STM32G474RE variant was used as the primary micro-controller. NUCLEO boards typically provide the ST-LINK interface, allowing for simplified firmware flashing and debugging.

The BIGTREETECH TMC2209 (V1.2) Stepper Driver was chosen for motor movement, given its quiet operation and smooth sine-wave generation. A 5-amp inline-fuse is installed between the TMC2209 motor voltage positive (+) and 24 VDC PSU positive (+), preventing over-current draw from the NEMA-17 devices.

To allow closed-loop feedback, each NEMA-17 was initially intended to have a MT6701 Magnetic Encoder module mounted on the back of the motor, with a dipole magnet installed on the motor shaft. Instead, only the final hand joint had a MT6701 mounted. The MT6701 could output a 12-bit analog signal of the current magnet rotation, and the signal would then be wired into a Analog-to-Digital Converter (ADC).

To convert the digital signal to a rotation amount in degrees (θ_i), the signal (d_s) is multiplied by the ratio $\frac{360}{2^n - 1}$, where n is the signal bit size.

$$\theta_i = d_s \cdot \frac{360}{2^n - 1} \quad (2)$$

3 Firmware Design

3.1 Libraries & Technologies

For safe and efficient embedded-firmware, the Rust programming language was used, given its memory safety and speed. The Embassy library was used for the Hardware Abstraction Layer (HAL), given the asynchronous-by-default model. This allowed stepper motion, UART communication, and feedback systems to be implemented easily without the complexity and runtime overhead of a Real Time Operating System (RTOS.)

3.2 UART Communication

The UART communication system was designed so that the TAAR system could receive a constant stream of GCODE bytes, while sending JavaScript Object Notation (JSON) formatted messages to communicating devices.

The communicating machine is intended to break a GCODE sequence into individual commands, sending one command block at a time. The firmware allocates 128 bytes of space to then read and parse the received block, sending a JSON response back. This JSON response can command the communicating

machine to continue sending the sequence, or end (due to errors or related issues in the received block.)

3.3 Motion Control

Each motor has a defined firmware implementation for motion. The motor motion is controlled by dynamically spaced pulses to the TMC2209 "STEP" pin, where the speed of the motor depends of the delay between each pulse.

To ensure all motors start and stop at the same time, the firmware calculates the delay for each motor. Larger movements yield shorter delays, with short movements yielding longer delays. Calculating the delay for each motor is done by first finding the largest required movement, A , then dividing A by each motor's required movement, θ_i , to obtain a normalized movement ratio. This ratio is then multiplied by the shortest delay, s , to determine the delay for each motor.

$$A = \max(\theta_1, \theta_2, \dots, \theta_n) \quad (3)$$

$$d_i = s \cdot \frac{A}{\theta_i} \quad (4)$$

4 Kinematics

We define the pair x, y, z as a point in a three-dimensional coordinate system. θ_b is defined as the base rotation, θ_s is defined as the shoulder rotation, θ_e is defined as the elbow rotation, and θ_h is defined as the hand rotation. The degree unit ($^\circ$) is used for rotation measurements, while x, y, z is in millimeters (mm).

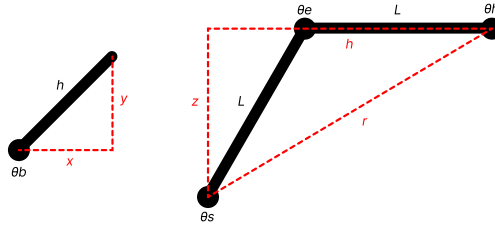


Figure 4: Base and arm on the described coordinate system

4.1 Inverse Kinematics

For a 4DOF arm, the Inverse Kinematic (IK) equations solve for base, shoulder, elbow, and hand angles given the pair x, y, z .

First, the base rotation, θ_b , is computed by the angle of a triangle drawn with x and y :

$$\theta_b = \arctan\left(\frac{y}{x}\right) \quad (5)$$

To obtain the shoulder and elbow angles, the hypotenuse (h) is computed for x and y , along with the shoulder and elbow described triangle hypotenuse (r).

$$h = \sqrt{x^2 + y^2} \quad (6)$$

$$r = \sqrt{h^2 + z^2} \quad (7)$$

The shoulder (θ_s) and elbow (θ_e) angles are then derived from the law of cosines:

$$\theta_s = \arctan\left(\frac{z}{h}\right) + \arccos\left(\frac{r}{2L}\right) \quad (8)$$

$$\theta_e = \arccos\left(1 - \frac{r^2}{2L^2}\right) \quad (9)$$

Lastly, the hand angle, θ_h , is described parallel to the ground.

$$\theta_h = -(\theta_s + \theta_e) \quad (10)$$

Where L is the common arm length

4.2 Forward Kinematics

For a 4DOF arm, the Forward Kinematic (FK) equations solve for the x, y, z coordinates given the base (θ_b), shoulder (θ_s), and elbow (θ_e) angles.

First, the radial distance from the base axis (r) and the vertical position (z) are computed by summing the contributions of each arm segment.

$$r = L \cos(\theta_s) + L \cos(\theta_s + \theta_e - 180) \quad (11)$$

$$z = L \sin(\theta_s) + L \sin(\theta_s + \theta_e - 180) \quad (12)$$

The radial distance is then projected onto the x and y axes using the base rotation.

$$x = r \cos(\theta_b) \quad (13)$$

$$y = r \sin(\theta_b) \quad (14)$$

Where L is the common arm length.

5 Conclusion

5.1 Cost

Table 2: Cost Breakdown

Component	Cost (\$USD)
PETG Plastic Spool - 1kg	16.25
5x NEMA-17 Stepper Motors	30.33
5x MT6701 Magnetic Encoder Modules	19.31
4x TMC2209 Stepper Driver Modules	16.73
24V DC Power Supply Unit	21.24
STMicroelectronics NUCLEO-G474RE	19.73
Arduino Style CNC Shield	25.40
608 Low-Friction Bearings	8.29
Misc. Wiring + Metric Screws	22.50
Total	\$179.78

No pre-determined budget was set for this project.

5.2 Accuracy

TAAR maintains an estimated accuracy of $\pm 3\text{mm}$ per movement. Considering a 3D-printed arm and the load of three 300g NEMA-17 stepper motors, this result is surprisingly accurate.

To better measure accuracy, commanding the end effector to a set of known Cartesian coordinates and measuring the resulting position with precision tools could yield a better estimate. The result above was obtained by locking each motor axis and "bumping it" with a small force to see the change in position, then measuring that change with a set of calipers.

5.3 Failures & Solutions

Table 3: Attempted Designs

Design	Failure	Solution
3D-Printed PCB	Precision machinery and issues with copper foil tape	Use an Arduino-style “CNC Shield” for mounting components
150mm Arm Length	Overloading stepper motors and causing weight issues	Change arm length to 100mm
MT6701 Joints	Angle wrapping with 6:1 gear ratio	Remove MT6701 on joints with a gear ratio
TMC2209 Excessive Heat	Over-current and lack of cooling	Larger heat-sink components and motion speed limits

5.4 Applications

The applications of TAAR are primarily educational. TAAR can showcase embedded firmware design for aspects like motor control and machine-to-machine communication.

The implementation of the JSON interface between devices is an example of USB control, where one device can send a GCODE stream, and TAAR can communicate back. The applications of a JSON protocol are limitless, as adding more instructions can be done by modifying the message headers.

The inverse and forward kinematics show applied mathematics and uses of trigonometry. TAAR is an example of physical systems ”broken down” into geometry.

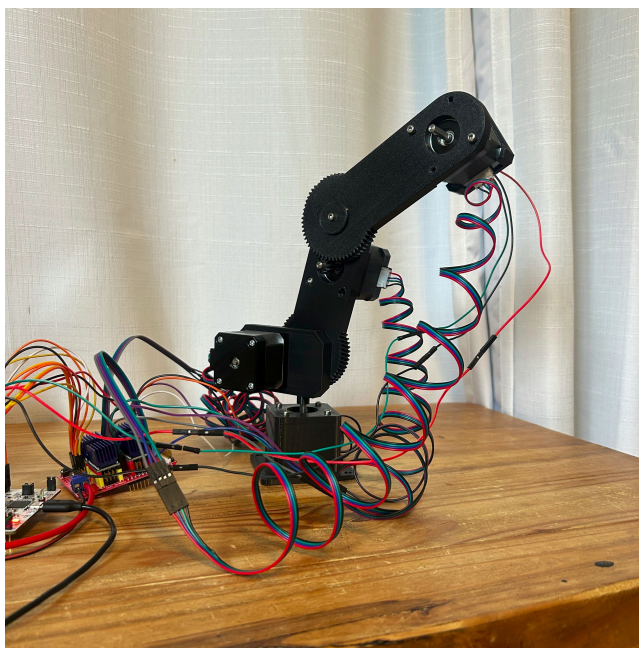


Figure 5: Completed TAAR assembly, August 13, 2026